R&T Semestre 2 2025/2026

SAE22 : Mesurer et caractériser un signal

Simulations numériques

Cette Situation Acquisition et d'Evaluation va traiter de réalisation ou acquisition de signaux numériques et de leurs exploitations. Elle se décompose en 4 parties :

- Partie 1 : Prise en main de Matlab
- Partie 2 : Calcul numérique d'une intégrale
- Partie 3 : Etude d'un signal temporel
- Partie 4 : Etude d'un canal Radio

Pour réaliser le travail demandé, vous disposez de :

- 8 séances de 2h encadrées
- 4 séances de 2h en autonomie à réaliser en salle à l'IUT.
- Du travail personnel à réaliser entre les séances.

L'évaluation se fera en contrôle continu sur la progression et vous aurez à produire :

- Un dossier de fichiers zip comprenant vos programmes commentés
- Un compte-rendu répondant aux différentes questions et analyses demandées.

Partie 1 : Prise en main de Matlab

I / Formation Matlab : « Onramp »

- Suivre les chapitres 1 (course Overview) à 9 (Plotting data) du module « Onramp » d'autoformation de Matlab : https://matlabacademy.mathworks.com/

II / Génération d'un signal sinusoïdal x(t)

On désire générer et afficher le signal $x(t) = 5 \times cos(2 \times \pi \times 1000 \times t)$ où x(t) est une tension en Volts.

Matlab étant un logiciel numérique, il va devoir **échantillonner** ce signal. On choisira ***fe*=40kHz** comme fréquence d'échantillonnage.

Le début du programme Matlab est le suivant :

```
1 -    f=1000;  %fréquence du signal sinusoïdal
2 -    A=5;  %amplitude de la sinusoïde
3
4 -    fe=40000; %fréquence d'échantillonnage
5 -    Te=1/fe;  %durée d'un échantillon
6 -    Ns=200; %nombre d'échantillons
7
8 -    t=0:Te:(Ns-1)*Te;
9
10 -   x=5*cos(2*pi*f*t);
```

Figure 1: début du programme de génération d'un cosinus

1) Que représente la variable t ? Quelle est sa nature ?

2) Que fait la ligne 10 ? Quelle est la nature de x ?

3) Combien de période du signal $x(t)$ seront calculées ?

4) Quelle petite critique ou amélioration peut être apportée à ce programme ?

- **Faire la saisie de ce programme** (Il est recommandé de travailler avec des « live script » qui est une forme de programmation qui permet de commenter et de mettre en forme le programme. De cette façon le compte-rendu de cette SAE sera partiellement fait). Nommer par exemple ce programme : P1sinus.mlx.

III / Affichage du signal x(t)

- Chercher de l'aide sur la fonction « plot » puis compléter le programme suivant afin d'afficher x(t) avec les informations suivantes :

  - Couleur : rouge
  - Mettre un titre : title
  - Mettre une légende : legend
  - Mettre le nom des axes sur la figure : xlabel et ylabel
  - Fixer les axes suivants : de 0 à 2ms et de -10V à +10V : axis

Il faut obtenir une figure similaire à la Figure 2 :

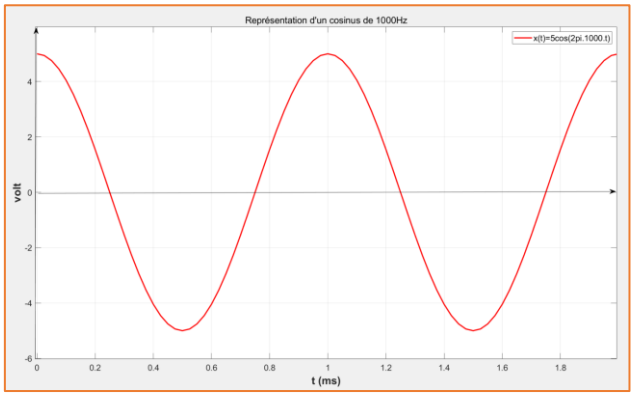


Figure 2 : affichage de la fonction cosinus x(t)

On veut maintenant créer un signal $y(t) = 3 \times sin(2 \times \pi \times 1000 \times t)$

- Compléter le programme afin de créer et d'afficher le signal $y(t)$ en bleu.

- Etudier la fonction « subplot » au moyen du menu « Help ».

- Modifier le programme afin d'afficher les deux courbes sur deux graphes sur la même figure et obtenir la représentation suivante :

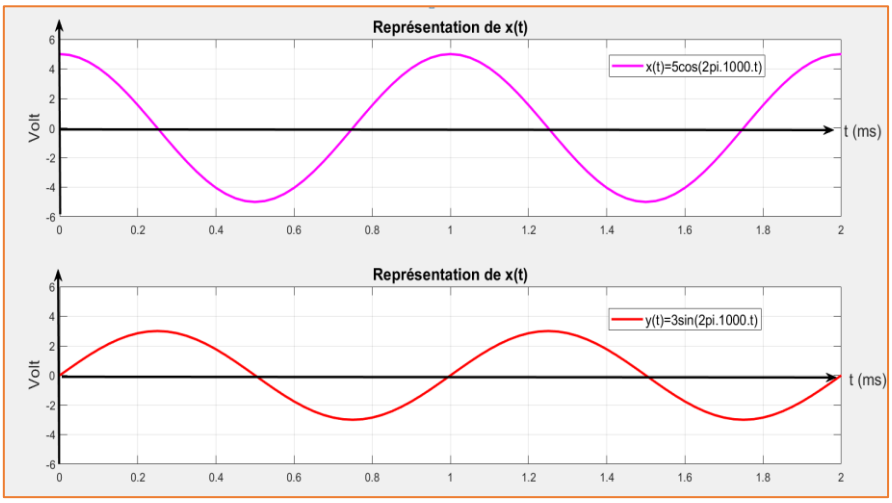


Figure 3 : représentation de deux signaux en utilisant la commande « subplot »

Il est parfois intéressant d'avoir les 2 représentations graphiques sur le même graphique.

- Rechercher comment afficher 2 courbes sur le même graphique avec la fonction « plot ».

- Compléter le programme pour obtenir la représentation suivante :

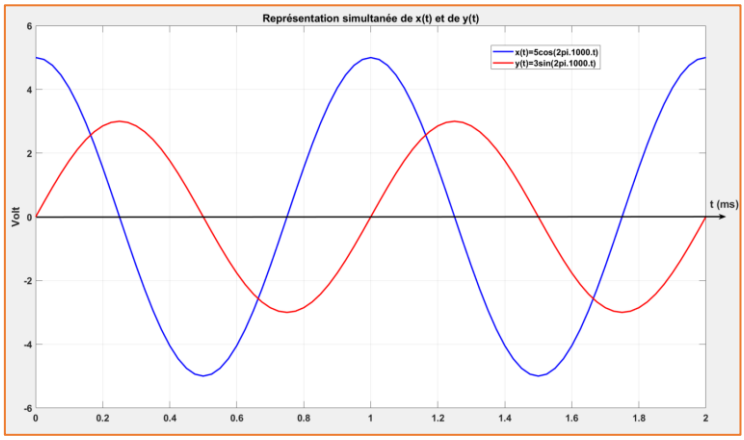


Figure 4 : représentation de deux signaux sur un même graphique

IV / Formation Matlab : suite

- Finir l'autoformation à Matlab Onramp.

V / Affichage du spectre

Matlab dispose d'une fonction FFT (Fast Fourier Transform) permettant de calculer la transformée de Fourier d'un signal.

Afin de faciliter le travail, une fonction, « spectre.m », permettant d'obtenir le spectre d'un signal, est fournie.

- Télécharger sur l'ENT puis décompresser le dossier « TP initiation Matlab.zip » contenant :
  - Le programme principal : « affichage_spectre_sinus.m »
  - Une fonction pour le calcul des spectres : « spectre.m »

1) Etudier la ligne 28 ainsi que la fonction « spectre.m » à laquelle elle fait appel.

```
27      %calcul de X(f): spectre en amplitude de x(t)
28 -    [X f]=spectre(x,fe,Ns);  % appel à la fonction  spectre.
```

2) Exécuter le script et valider le résultat.

Remarque : par la suite vous pourrez réutiliser cette fonction « spectre.m » pour tracer le spectre de n'importe quel signal.

- Créer une fonction « spectredBV.m » afin d'afficher maintenant le spectre en dBV ou dBµV. On utilisera pour cela la fonction $log10(X)$. On ne calculera pas ici la valeur efficace pour le moment.
- Compléter le programme pour tracer le spectre en dBV.
- Valider le résultat obtenu.

✋ Réaliser l'affichage comme montré ci-dessous, en mieux (penser aux noms des axes et autres …)

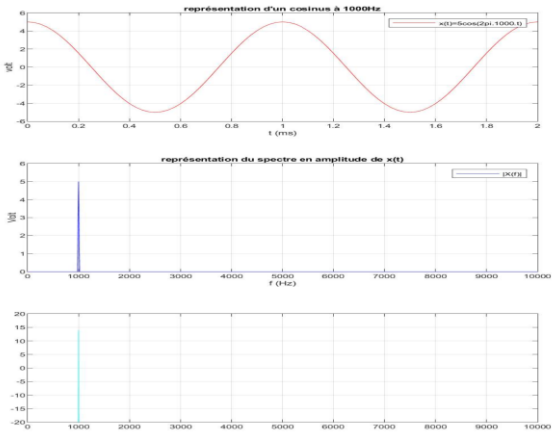


Figure 5 : représentation du signal x(t), de son spectre et de ce spectre en dBV

VI / Multiplication de deux signaux sinusoïdaux

On veut à présent réaliser la multiplication de deux signaux sinusoïdaux. En effet cette étape est fondamentale dans la transmission de signaux car elle réalise une transposition de fréquence. Ceci sera vu à l'aide du spectre de ce produit.

Les caractéristiques de l'échantillonnage seront les suivantes :

- Fréquence d'échantillonnage : *fech*=40kHz
- Nombre d'échantillons : Ns=2000

Les signaux à multiplier sont les suivants :

- $x1(t) = A1 \times cos(2 \cdot \pi \cdot f_1 \cdot t)$ avec A1=5V et *f1*=1kHz
- $x2(t) = A2 \times cos(2 \cdot \pi \cdot f_2 \cdot t)$ avec A2=3V et *f2*=3kHz

✋ Générer les deux signaux $x1(t)$ et $x2(t)$.

1) Vérifier la taille des vecteurs $x1$ et x2 créés. Justifier.

2) Quelle est la différence entre les opérations suivantes sous Matlab : « ***** » et « **.*** » ? Quel est l'opérateur à utiliser pour la multiplication demandée ?

3) Quelle est l'opération réalisée avec l'opérateur « .* » ? C'est-à-dire, en fonction des indices, quels sont les termes multipliés entre eux ? (z(1)=x1(??)*x2(??) ….)

4) Calculer *z* le produit de ces deux signaux : $\boldsymbol{z = x1 .* x2}$

5) Quelle est la dimension du vecteur z ? Justifier.

✋ Afficher sur une même figure, mais sur trois graphes les chronogrammes de $x1(t)$, $x2(t)$ et $z(t)$.

✋ Calculer les spectres de ces trois signaux et afficher les sur trois graphes d'une nouvelle figure.

Les résultats obtenus doivent être de la forme suivante :

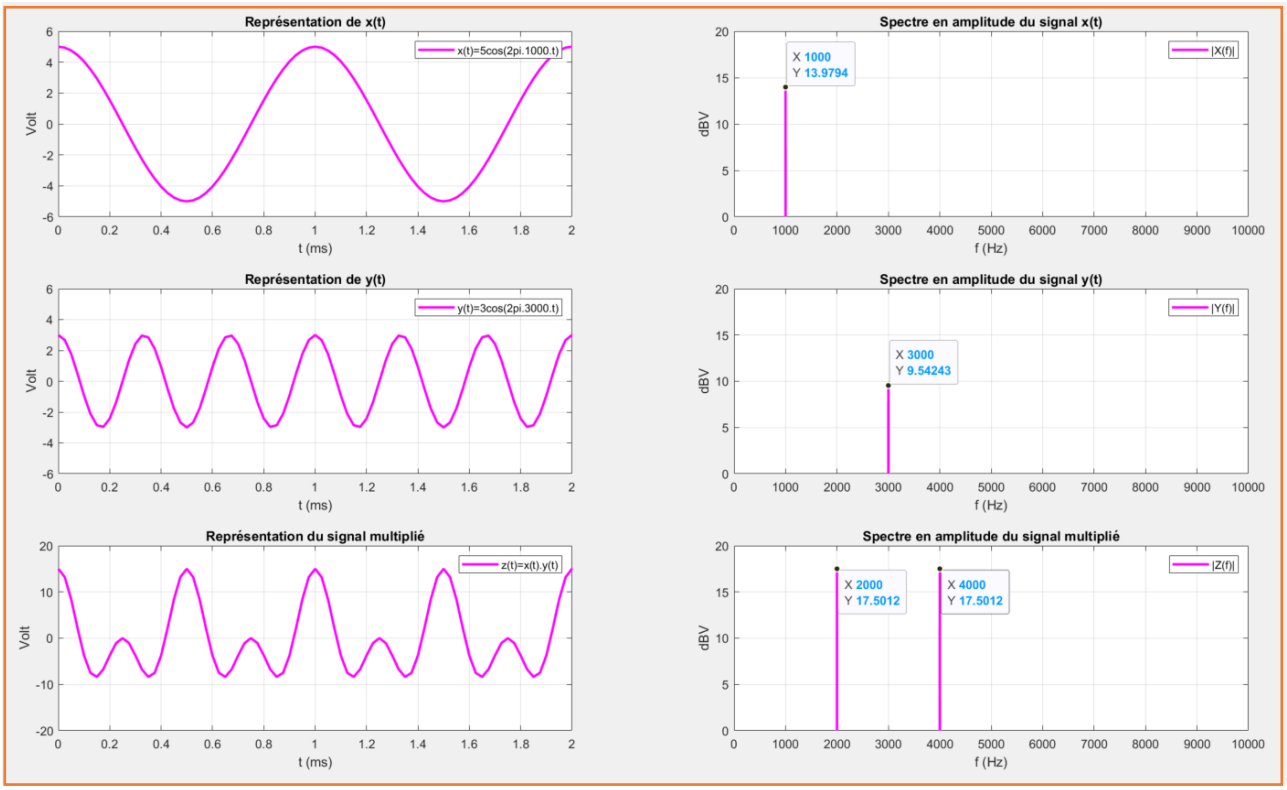


Figure 6: multiplication de deux signaux sinusoïdaux

6) Analyser les résultats obtenus et comparer avec les spectres attendus théoriquement.

7) Recommencer en prenant maintenant $x1(t) = x2(t) = 3 \times cos(2 \cdot \pi \cdot 1000 \cdot t)$. Analyser. Quelle est la fonction réalisée ?!

VII / Somme des éléments d'un tableau

L'objectif est de vous familiariser avec la fonction sum() de Matlab. Cette fonction sera fréquemment utilisée par la suite…

- Saisir, commenter puis exécuter le programme suivant :

```
clear all;
close all;
clc;

x=(1:100);

S=sum(x);
S2=sum(x(2: end-2));
```

- Justifier les valeurs obtenues pour S et S2.

Partie 2 : Calcul numérique d'une intégrale

I / Présentation du contexte

Soit $f(t)$ une fonction continue définie sur un intervalle $t \in [a;b]$. On souhaite évaluer numériquement l'intégrale : $I = \int_a^b f(t) \cdot dt$

Pour donner une valeur approchée, on prendra $\boldsymbol{n}$ **échantillons** sur la courbe uniformément répartis dans l'intervalle $[a;\ b]$. On va donc découper $[a;\ b]$ en $\boldsymbol{n-1}$ **sous-intervalles** de longueur $\boldsymbol{dt = \frac{(b-a)}{n-1}}$ (cf. Figure 1).

Le i[ième] échantillon sera ainsi à l'abscisse : $\boldsymbol{t_i = a + (i-1).dt}$ où i est un nombre entier compris entre 0 et n , ce qui s'écrit : $i \in \langle 1\ ;\ n \rangle$.

Les méthodes de calcul numérique d'intégrales consistent à approximer, sur chaque sous-intervalle, la fonction $f(t)$ par un polynôme.

Dans ce qui suit, on va essayer d'évaluer numériquement l'intégrale : $\boldsymbol{I = \int_0^1 e^{2t} \cdot dt}$

L'objectif est d'évaluer cette intégrale suivant plusieurs méthodes puis de comparer ces méthodes en étudiant leurs vitesses de convergence.

II / Méthode des rectangles à gauche

La méthode des rectangles à gauche consiste à remplacer sur chaque sous-intervalle $J_i = [t_i\ ;\ t_{i+1}]$, la fonction $f(t)$ par $f(t_i)$.

La Figure 1 illustre le calcul de $I = \int_0^1 e^{2t} dt$ par la méthode des rectangles avec **n=5 échantillons**, soit $\boldsymbol{n-1 = 4}$ **intervalles**…

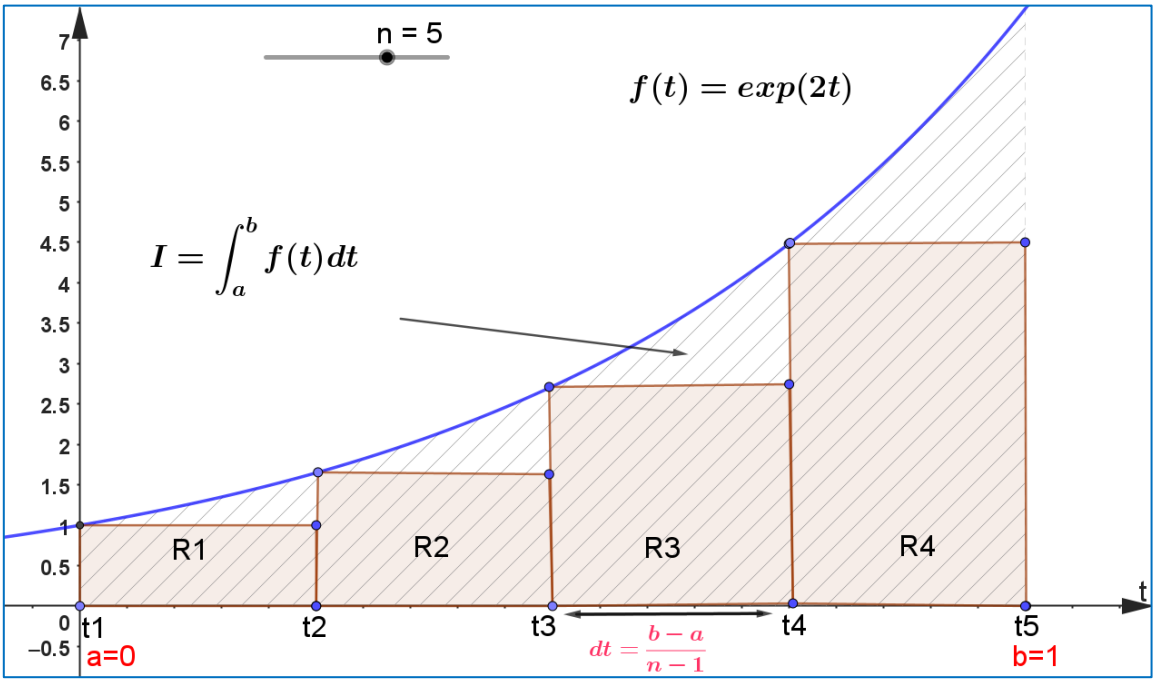


Figure 1: intégration par la méthode des rectangles

Selon cette méthode, la valeur approchée de I avec n=5 points vaut $R = R_1 + R_2 + R_3 + R_4$.

Soit : $R = f(t_1) \times dt + f(t_2) \times dt + f(t_3) \times dt + f(t_4) \times dt$ (eq 1)

Préparation théorique :

1) Calculer théoriquement la valeur de I. Faire une application numérique à 10^{-5} près.

2) Pour **n=5 échantillons**, calculer R à l'aide d'un tableur (valeur approchée de I selon la méthode des rectangles) (eq 1).

3) Montrer que de manière générale la formule de l'approximation de l'intégrale I par la méthode des rectangles avec n échantillons est :

$$R = \frac{b-a}{n-1} \sum_{i=1}^{n-1} f(t_i)$$

Réalisation :

On se propose maintenant d'implanter le calcul de cette intégrale en utilisant Matlab.

- **Compléter le script suivant** afin d'implanter l'approximation de la méthode des rectangles sur Matlab.

```
clear all;
close all;
clc;
%% Définition de la fonction et des bornes d'intégration
f = @(x) exp(2*x);  % fonction étudiée
a = 0;
b = 1;

%% Echantillonnage de la fonction sur n points
n = 11;
dt = (b-a)/(n-1); % largeur d'un intervalle.
t = a+(0:n-1)*dt;
y = f(t);
plot(t,y,'-bo','LineWidth',2);

%% méthode des rectangles
% à compléter...
```

- Valider le résultat pour $n = 5$.

- Calculer l'erreur relative en % dans votre programme. Relever cette erreur pour $n = 1001$.

III / Méthodes des trapèzes

On se rend compte que la méthode des rectangles peut être améliorée. La méthode des trapèzes consiste à approximer la fonction $f(t)$ dans chacun des sous intervalles par des segments. La figure 2 illustre cette méthode pour $n = 5$ échantillons soit 4 intervalles.

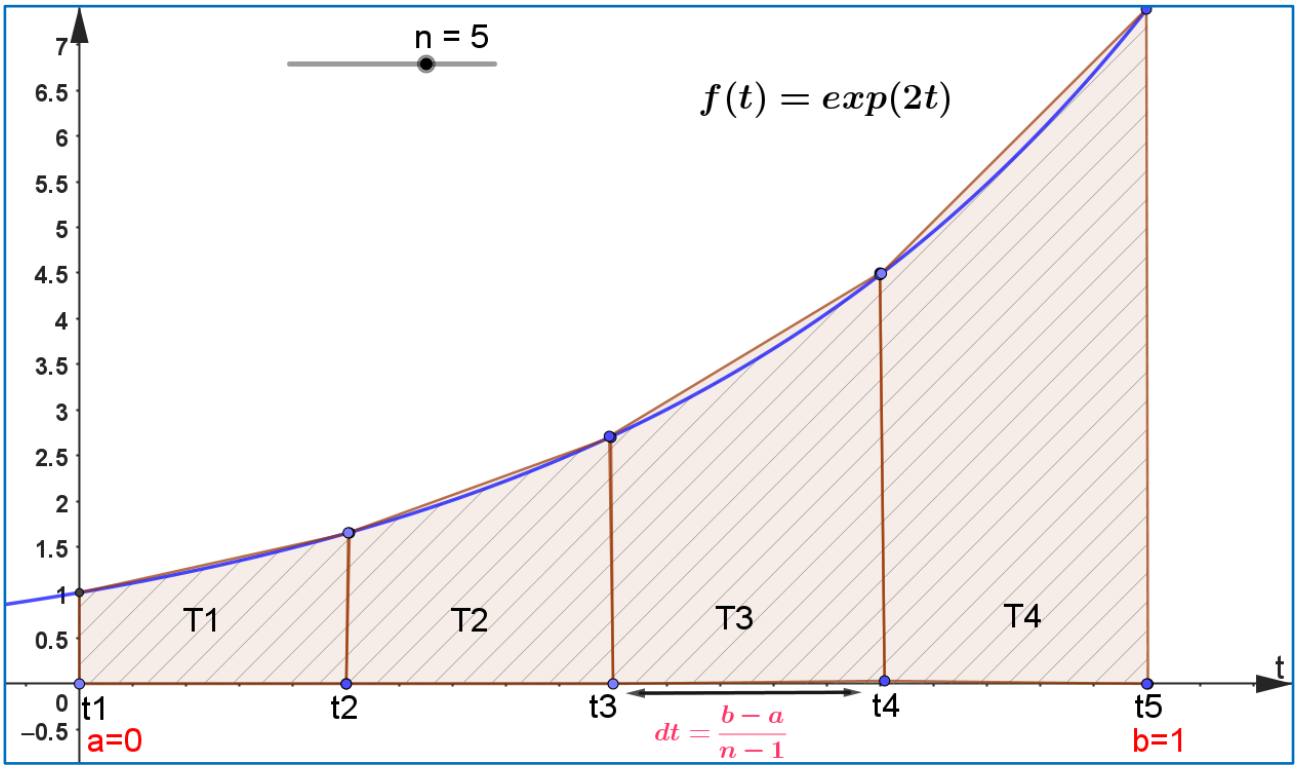


Figure 2: intégration par la méthode des trapèzes

Préparation théorique :

1) Calculer l'aire du trapèze $T1$, sur l'intervalle $J_1 = [t_1 \; ; \; t_2]$.

2) De façon plus générale, que vaut l'aire du trapèze Ti dans l'intervalle $J_i = [t_i \; ; \; t_{i+1}]$ pour $i \in \langle 1 \; ; \; n-1 \rangle$? Exprimer le résultat en fonction de $dt = \frac{b-a}{n-1}$, $f(t_i)$ et $f(t_{i+1})$

3) Montrer que la formule du calcul approché de l'intégrale I par la méthode des trapèzes est :

$$T = \frac{b-a}{n-1} \sum_{i=1}^{n-1} \frac{f(t_i) + f(t_{i+1})}{2} \qquad (eq\ 2)$$

4) (*) Montrer que cette formule peut se mettre sous la forme suivante :

$$T = \frac{b-a}{n-1} \left(\frac{f(t_1) + f(t_n)}{2} + \sum_{i=2}^{n-1} f(t_i) \right)$$

5) Pour $n = 5$, calculer à l'aide d'un tableur la valeur approchée de I, nommée T, par la méthode des trapèzes.

Réalisation :

- **Implanter** la méthode des trapèzes sur Matlab.
- Valider le résultat pour $n = 5$.
- Calculer l'erreur relative en % dans votre programme. Relever cette erreur pour $n = 1001$.

IV / Méthode de Simpson

C'est une des méthodes numériques des plus performantes.

- En faisant des recherches sur Internet (par exemple sur le site https://fr.wikipedia.org/wiki/M%C3%A9thode_de_Simpson) , expliquer le principe de cette méthode.

- **Facultatif :** implanter cet algorithme. Comparer l'erreur relative pour n=1001 points.

V / Facultatif : Méthode de Monte-Carlo (*)

Principe de la méthode de Monte Carlo :

Cette méthode consiste dans un premier temps à considérer un rectangle (en bleu sur la figure 3) qui inclue la surface de l'intégrale à calculer. La surface S_R du rectangle est facile à calculer.

On génère aléatoirement n points dans ce rectangle.
Parmi ces n points, on détermine le nombre de points inclus dans la zone d'intégration (en rouge). On note $n1$ ce nombre de points.
L'aire de la surface est approximée par : $\boldsymbol{I_{Monte\ Carlo} = \frac{n1}{n}.S_r}$

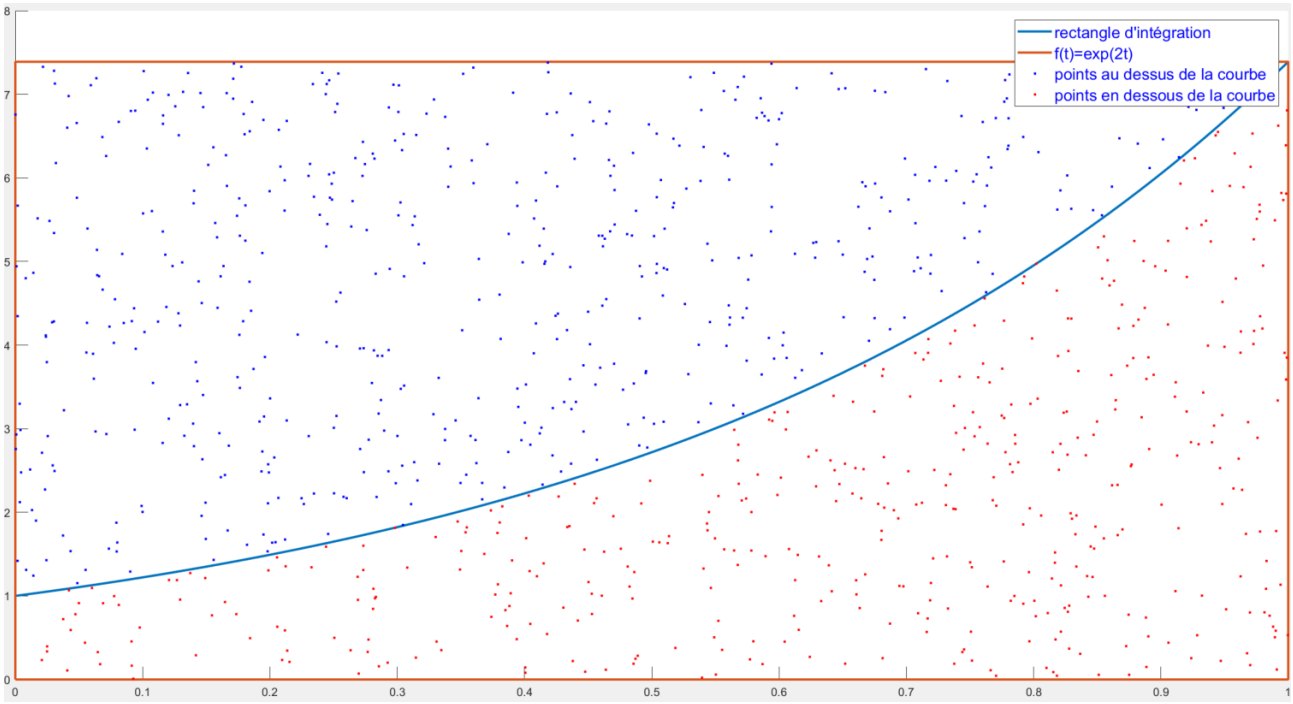


Figure 3: intégration par la méthode de Monte Carlo

Réalisation :

- Implanter le calcul de l'intégrale I en utilisant la méthode de Monte-Carlo

Remarque : cette méthode est particulièrement adaptée au calcul d'intégrales multidimensionnelles (volumes…). En 2D, elle a une vitesse de convergence très faible.

- Calculer l'erreur relative en % dans votre programme. Relever cette erreur pour $n = 1001$.

VI / Vitesse de convergence (*)

Dans cette partie, on prendra toujours $I = \int_0^1 e^{2t} dt$.

On s'intéresse aux erreurs d'approximations de l'Intégrale I en fonction du nombre de points, n.

On note : $m_1 = \sup_{[a,b]} |f'(t)|$, la valeur maximale de f' sur l'intervalle $[a, b] = [0,1]$.

De même on note, $m_2 = sup_{[a,b]} |f''(t)|$

1) Déterminer la formule de $f'(t)$ et $f''(t)$.

2) Montrer que : $m_1 = 2e^2$ et $m_2 = 4e^2$

Soit ***Rn*** **et** ***Tn*** les approximations sur ***n points*** de l'intégrale I respectivement par la méthode des rectangles et celle des trapèzes.

On donne les formules de majorations suivantes :

$$|I - Rn| \leq \frac{(b-a)^2}{2(n-1)} m_1 \quad (1)$$

$$|I - Tn| \leq \frac{(b-a)^3}{12(n-1)^2} m_2 \quad (2)$$

3) Pour n=1001, justifier que les résultats obtenus dans les questions précédentes par simulation vérifient les majorations (1) et (2).

4) Pour la méthode des rectangles, faire un programme permettant de représenter l'évolution de l'erreur $|I - R|$ en fonction de n pour n variant de 3 à 1000 (par pas de 2). On pourra prendre une échelle logarithmique.

5) Représenter aussi l'évolution en fonction de n de la majoration $\frac{(b-a)^2}{2(n-1)} m_1$. Conclure

6) Faire la même étude pour la méthode des trapèzes.

7) **Facultatif :** recommencer avec la méthode de Simpson. La formule théorique est donnée par : $|I - S| \leq \frac{(b-a)^5}{180(n-1)^4} m_4 \quad avec \ m_4 = sup_{[a,b]} |f^{(4)}(t)|$

8) **Facultatif :** faire aussi l'étude pour la méthode de Monte Carlo.

9) **Synthèse :** faire un tableau comparatif sur les performances des méthodes étudiées.

Partie 3 : Etude d'un signal temporel

Objectif : on se propose d'analyser sous Matlab un signal acquis à l'aide d'un oscilloscope. On souhaite le caractériser en effectuant les calculs suivants :

- Mesure de l'amplitude de sa tension.
- Calcul de la valeur moyenne et de la valeur efficace de sa tension.
- Détermination du spectre en amplitude.
- Détermination du fondamental et des harmoniques.
- Calcul de la puissance moyenne en utilisant Parseval.

I / Acquisition des signaux

- Générer à l'aide d'un GBF un signal $x(t)$, sinusoïdal de fréquence 1000Hz et d'amplitude 5V.
- Observer ce signal à l'oscilloscope et le capturer à l'aide du logiciel « Keysight Benchvue ».
- Exporter ce signal en fichier Matlab. Pour cela, aller sur « ***Données de trace*** » puis « ***Exporter*** » (cf. Figure 1). On nommera le fichier exporté « **sinus.mat** ».

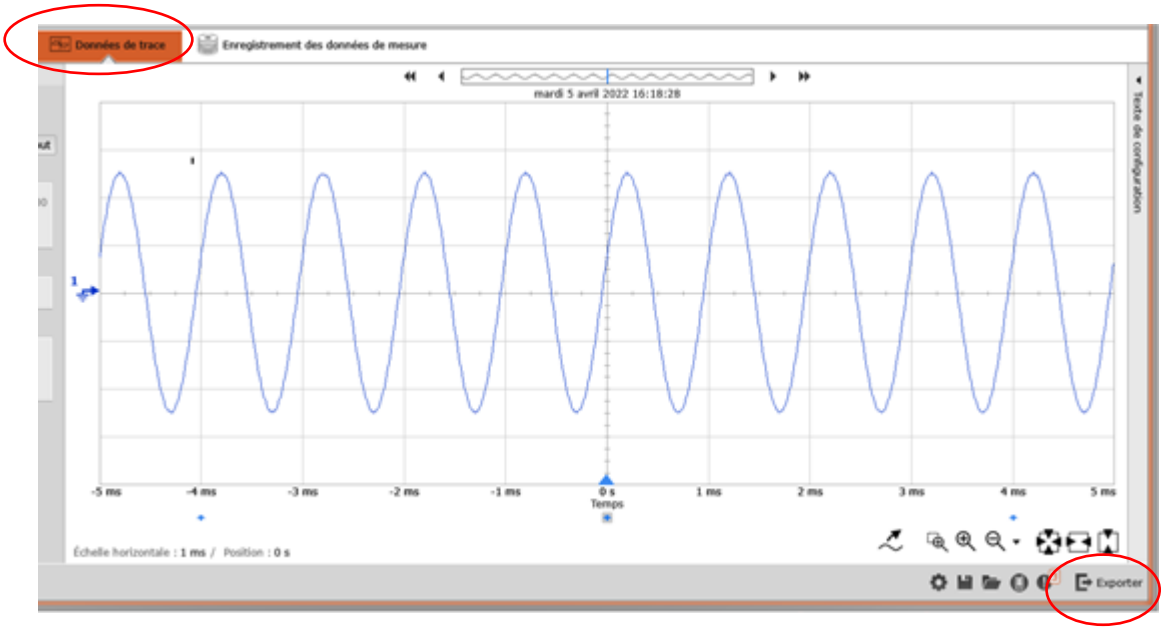
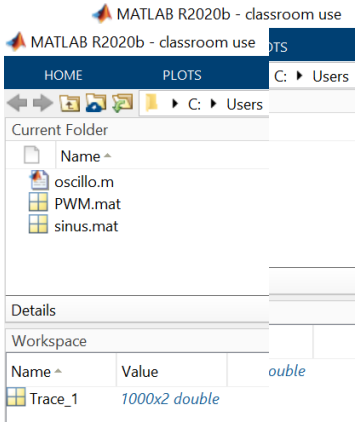


Figure 1 : export du signal de l'oscilloscope

- Recommencer en générant maintenant y(t) : signal carré 0-5V de fréquence 100kHz et de rapport cyclique $R_c = 0{,}2$. Le fichier importé sera nommé « **PWM.mat** ».

II / Etude du signal sinusoïdal : $x(t)$

II.1 / Extraction et affichage



- Créer un nouveau script Matlab, nommé « oscillo.m » et l'enregistrer dans un répertoire identifié.
- Dans ce répertoire ajouter les fichiers « sinus.mat » et « PWM.mat » importés précédemment.
- Récupérer les données avec la procédure suivante :

```
close;
clear;
clc;

load sinus.mat
% load PWM.mat
temp=transpose(Trace_1);
t=temp(1,:);
x=temp(2,:);
```

1) Quel est le rôle de la commande « transpose » ?

- Afficher le signal $x(t)$. (Commande plot).
- Récupérer le nombre d'échantillons, Ns, de ce signal à l'aide de la commande « size ».
- A partir du tableau des temps, calculer la période d'échantillonnage : Te.

Remarque : Ns et Te seront utiles pour la suite…

II.2 / Exploitation des données du signal x(t)

II.2.a / Calcul de l'amplitude

- En utilisant les fonctions « max » et « min » calculer l'amplitude de ce signal.

II.2.b / Représentation du spectre en amplitude et détermination du fondamental

- En utilisant la fonction « spectre » vue précédemment, calculer puis afficher le spectre en amplitude de ce signal x(t).
- A l'aide de la fonction « max » déterminer l'amplitude du fondamental de x(t).
- Regarder l'aide de la fonction « max » et retrouver l'indice, dans le tableau de x, où se trouve ce maximum.
- En déduire une détermination de la valeur de la fréquence du fondamental : F_0.

II.2.c / Calcul de la valeur moyenne

La valeur moyenne du signal est donnée par la formule : $x_{moy} = \frac{1}{T} \times \int_0^T x(t) \cdot dt$ où T représente la période de la *composante fondamentale*.

- A partir de la fréquence du fondamental, déterminer la période du signal : $T = \frac{1}{F_0}$
- En déduire le nombre d'échantillons pour une période de x(t) à l'aide de la fonction « round » : $Nsparperiode = M = round\left(\frac{T}{Te}\right)$
- Réaliser un programme Matlab qui calcule la valeur moyenne du signal en utilisant la méthode des rectangles.

II.2.d / Calcul de la valeur efficace

La valeur efficace du signal est donnée par la formule : $x_{eff} = \sqrt{\frac{1}{T} \times \int_0^T x^2(t) \cdot dt}$

- Réaliser un programme Matlab qui calcule la valeur efficace du signal en utilisant la méthode des rectangles.

III / Etude du signal PWM : y(t)

III.1.a / Etude théorique du signal PWM

On considère le signal y(t) suivant :

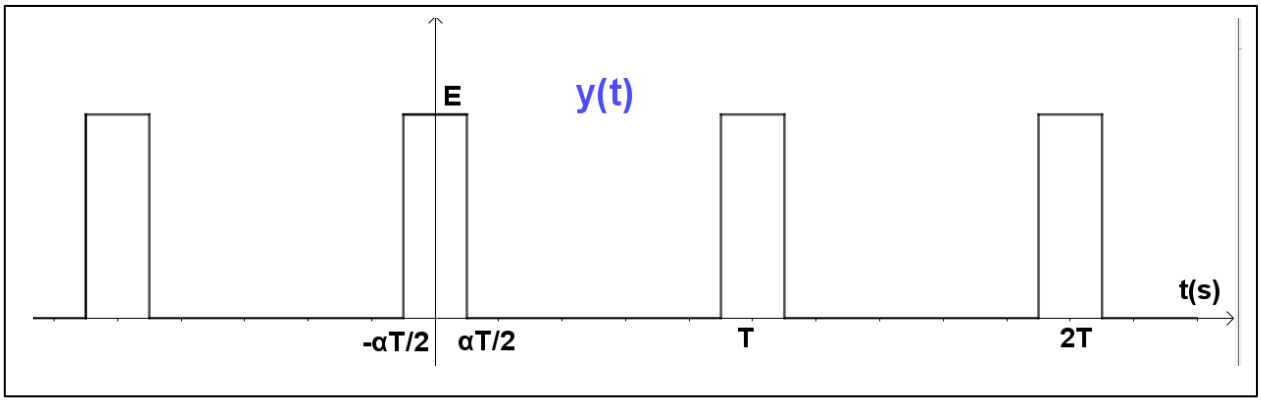


Figure 2: signal PWM

1) Calculer **théoriquement** la valeur moyenne et la valeur efficace de ce signal.

On admet que la décomposition en série de Fourier de ce signal est :

$$y(t) = \alpha E + \frac{2E}{\pi}\sin(\alpha\pi).\cos(wt) + \frac{2E}{2\pi}\sin(2\alpha\pi).\cos(2wt) + \frac{2E}{3\pi}\sin(3\alpha\pi).\cos(3wt) + ...$$

Soit en utilisant le signe somme : $y(t) = \alpha.E + \sum_{n=1}^{+\infty} \frac{2.E}{n.\pi}\sin(n.\alpha.\pi).\cos(n.\omega.t)$

Pour les applications numériques, on prendra $\boldsymbol{E = 5V}$; $\boldsymbol{\alpha = 0,2}$; $\boldsymbol{\omega = 2\pi.100.10^3\ rad/s}$.

2) Calculer à l'aide d'un tableur la valeur numérique de la composante continue ainsi que l'amplitude du fondamental, et des harmoniques 2, 3 et 4.

3) Représenter avec le tableur le spectre en amplitude de ce signal jusqu'à $f = 400kHz$.

4) En utilisant Parseval, déterminer la puissance de ce signal (on se limitera à l'harmonique 4).

III.1.b / Etude sous Matlab du signal PWM

- Réaliser un programme Matlab qui calcule la valeur moyenne, la valeur efficace et le spectre du signal y(t) importé de l'oscilloscope.

- Comparer les résultats avec ceux calculés dans la partie théorique.

- Retrouver les amplitudes de la composante continue, du fondamental et des $N = 14$ premiers harmoniques (on pourra utiliser la fonction maxk).

- Appliquer le théorème de Parseval afin de déterminer la puissance de ce signal.

- Conclure.

Partie 4 : Etude d'un canal Radio

Objectif : lors du TPn°3 des TP de Télécoms, vous avez acquis le spectre en réception d'une transmission radio. Cette transmission été effectuée autour d'une fréquence porteuse de f_p=27MHz.

On se propose d'analyser sous Matlab ce spectre et de la caractériser en effectuant les calculs suivants :

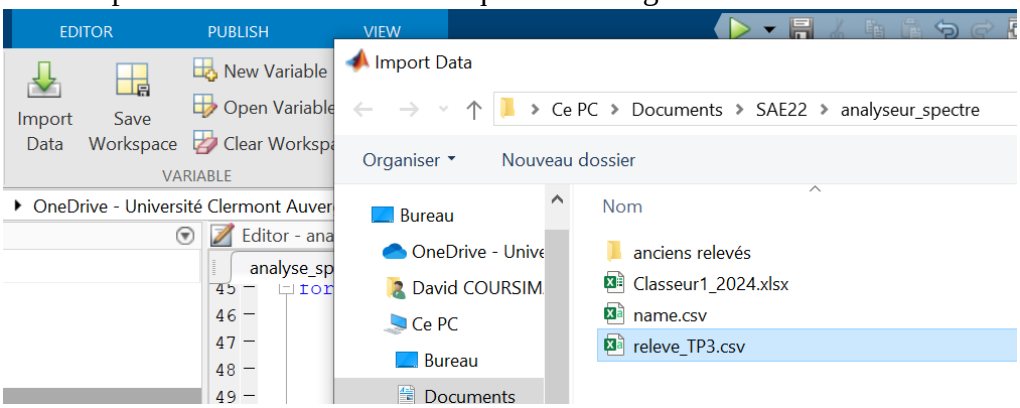
- Calcul de la bande occupée à 20dB
- Calcul de la densité spectrale de puissance (DSP)
- Détermination de la puissance totale du canal

I / Récupération des données

I.1 / Importation des données sur Matlab

Il faut importer les données sauvegardées pendant la séance de TP de Télécoms sur Matlab.

- Enregistrer le fichier dans le répertoire de votre projet Matlab.
- Importer le fichier comme indiqué dans la figure suivante :



- Sélectionner les données à importer puis choisir le type de sortie 'column vector'

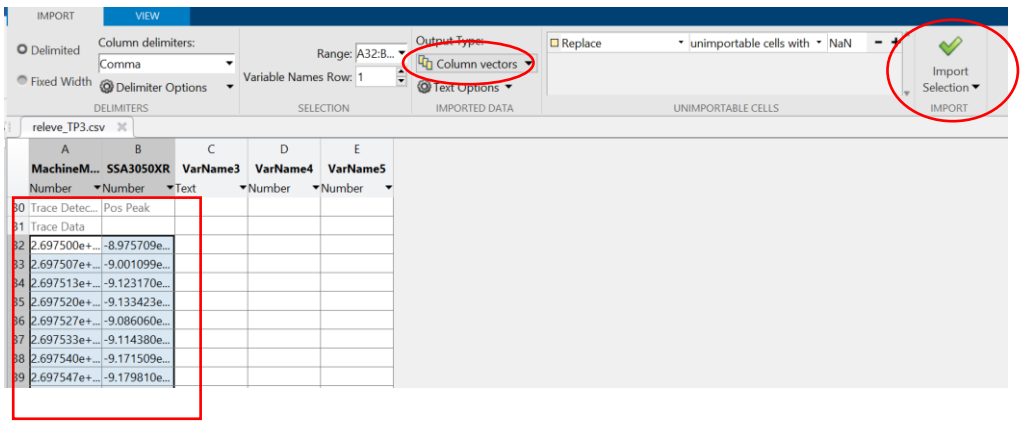


Figure 3: importation des données sur Matlab

- Importer ensuite les données
- Deux vecteurs colonnes ont dû être crées dans le Workspace. Cliquer sur ces vecteurs pour visualiser leurs contenus…

I.2 / Visualisation des données

- Créer un script permettant d'afficher l'évolution de la puissance en fonction de la fréquence.

II / Lissage des données

Les données récupérées sont relativement bruitées. On va essayer de réduire ce bruit en appliquant un filtre appelé filtre de lissage ou filtre moyenneur. L'algorithme est le suivant :

Entrées :
P_{dBm} % tableau des puissances en dBm.
N_s % nombre de cases de ce tableau
K =4 % ordre du filtre (on fera la moyenne sur 2K+1 échantillons)

Initialisation :
Créer un tableau $P_{L\ dBm} = P_{dBm}$ % Ce tableau servira à stocker les données lissées.

Traitement :
Pour n allant de K+1 à N_s-K

$$P_{L\,dBm}(n) = \frac{\sum_{k=-K}^{K} P_{dBm}(n+k)}{2K+1}$$

fin

Sortie : $P_{L\ dBm}$

- Implanter cet algorithme et afficher le résultat.
- Faire varier l'ordre du filtre de lissage (paramètre K) et visualiser son effet.

III / Détermination de la bande occupée à $-20dB$

- En utilisant la fonction max, déterminer le niveau maximal de puissance dans ce relevé ainsi que la fréquence correspondante. Aide : la fonction « max » renvoie le maximum d'un tableau mais aussi l'indice correspondant…
- Compléter le programme afin de déterminer la bande occupée de ce canal à $-20dB$.

IV / Calcul de la puissance du canal

- Convertir le tableau des puissances, PdBm, en un tableau exprimé en Watt : P_W.
- En déduire un tableau des densités spectrales de puissances : $DSP = \frac{P_W}{RBW}$.
- Calculer la puissance du canal comme :

$$P_{canal} = \int_{f_{min}}^{f_{max}} DSP(f).df$$

V / Présentation des résultats

- Utiliser les commandes « strcat » et « text » afin d'afficher les calculs de la bande occupée à $20dB$ et de la puissance canal.

On devra avoir un résultat analogue à celui de la Figure 4.

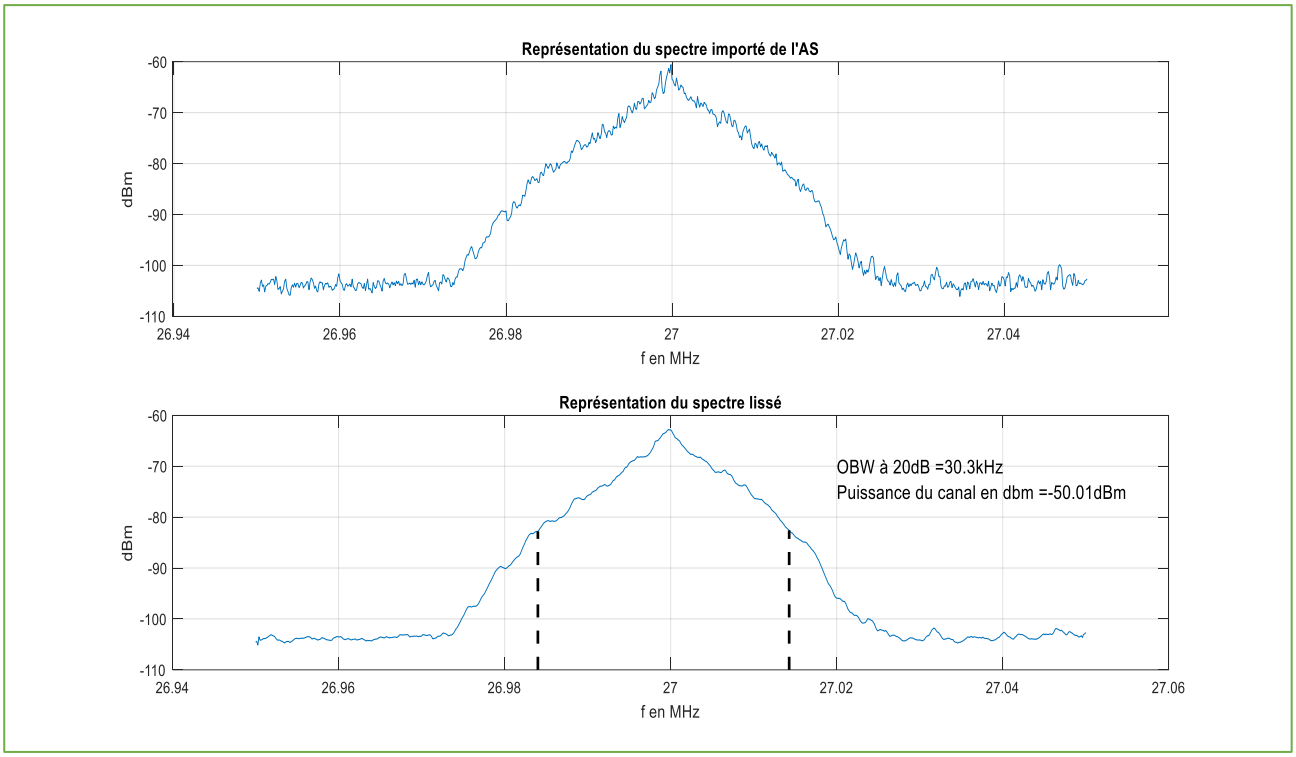


Figure 4: résultat final attendu